

ITM Skills University

School of Engineering and Technology
B.Tech – Computer Science and Engineering

PROJECT REPORT

Data Structures and Algorithms (C++)

NATIONAL BLOOD BANK COORDINATION NETWORK

STUDENT NAME	RASHMEET KAUR NANADE
ROLL NUMBER	150096725165
COHORT	DEMIS HASSABIS
COURSE	B.TECH CSE - SEMESTER II
SUBJECT	DATA STRUCTURES & ALGORITHMS (C++)
UNIVERSITY	ITM SKILLS UNIVERSITY
ACADEMIC YEAR	2025–29

Academic Year 2025–29

ABSTRACT

Blood is one of the most critical medical resources required for surgeries, emergency treatments, accident victims, cancer therapies, and disaster management operations. Efficient management of blood inventories, donor records, hospital requests, and transportation logistics is essential for ensuring timely medical support and saving lives.

The National Blood Bank Coordination Network is a healthcare logistics management system developed using C++ and Data Structures and Algorithms. The system provides centralized coordination among blood banks, hospitals, donors, and emergency response units. It enables efficient blood inventory management, donor search operations, emergency request processing, route optimization, and healthcare analytics reporting.

The project incorporates several important Data Structures and Algorithms. Queue is used for handling emergency blood requests in First-In-First-Out order. Merge Sort is implemented to rank blood banks according to inventory availability. Binary Search is used to rapidly locate donor records in sorted databases. Dijkstra's Algorithm identifies the shortest transportation routes between blood banks and hospitals. Breadth First Search (BFS) verifies network connectivity among healthcare facilities.

The system demonstrates how Data Structures and Algorithms can be applied in real-world healthcare environments to improve operational efficiency, reduce response times, optimize resource allocation, and enhance decision-making processes.

Keywords :- Blood Bank Management, Healthcare Logistics, Queue, Merge Sort, Binary Search, Dijkstra's Algorithm, BFS, C++, Data Structures and Algorithms.

INTRODUCTION

Blood is one of the most essential resources in modern healthcare systems. It plays a critical role in surgeries, trauma care, cancer treatments, organ transplants, maternity services, and emergency medical interventions. Every day, hospitals and healthcare facilities require a continuous supply of different blood groups to meet patient demands. Any delay in blood availability can lead to severe consequences and may even result in loss of life. Therefore, efficient management of blood resources is an important responsibility for healthcare organizations.

Traditional blood bank management systems often rely on manual processes or isolated databases that operate independently. Such systems may face challenges related to inventory monitoring, donor tracking, emergency request handling, and communication between hospitals and blood banks. During emergencies, the inability to quickly identify available blood units or suitable donors can significantly delay treatment. These limitations highlight the need for an integrated and intelligent blood bank coordination platform.

The National Blood Bank Coordination Network is designed to address these challenges by creating a centralized healthcare logistics system that connects blood banks, hospitals, donors, and transportation networks. The system enables real-time visibility of blood inventories, facilitates efficient donor searches, processes

emergency requests systematically, and supports optimal distribution of blood resources. By providing a coordinated framework, the system improves operational efficiency and ensures better utilization of available blood supplies.

Data Structures and Algorithms play a vital role in the development of such a healthcare management system. Different algorithms are required to perform specific operations efficiently. Queue is used to manage emergency blood requests in the order they are received, ensuring fair and organized processing. Binary Search enables rapid identification of donor records within sorted databases. Merge Sort is utilized to rank blood banks according to inventory availability, allowing healthcare administrators to make informed allocation decisions.

The project also incorporates graph-based algorithms to improve transportation and distribution operations. Blood banks and hospitals are represented as nodes within a transportation network, while road connections are represented as weighted edges. Dijkstra's Algorithm is used to determine the shortest route between facilities, reducing delivery time during emergencies. Breadth First Search (BFS) is employed to verify network connectivity and ensure that all healthcare facilities remain accessible within the distribution network.

The National Blood Bank Coordination Network demonstrates the practical application of Data Structures and Algorithms in solving real-world healthcare challenges. Through efficient inventory management, donor coordination, emergency request processing, route optimization, and analytical reporting, the system contributes to faster decision-making and improved healthcare logistics. The project highlights how appropriate algorithm

selection can enhance system performance and support life-saving medical operations in a scalable and reliable manner.

PROBLEM STATEMENT

2.1 CONTEXT

Blood is a life-saving medical resource that is required continuously in hospitals, trauma centers, surgical units, maternity wards, cancer treatment facilities, and emergency response departments. Every day, thousands of patients depend on the availability of blood for successful treatment and recovery. Maintaining an adequate blood supply is therefore one of the most important responsibilities of healthcare systems worldwide.

However, managing blood resources is a complex task. Blood banks must continuously monitor inventory levels, track donor availability, process hospital requests, and coordinate transportation activities. Since blood has a limited shelf life, improper inventory management can lead to shortages or wastage. During emergencies, delays in locating suitable blood units can significantly affect patient outcomes.

With increasing healthcare demands and growing populations, traditional manual systems are no longer sufficient for handling large-scale blood management operations. Modern healthcare organizations require intelligent and efficient digital solutions that can coordinate blood resources across multiple facilities while ensuring timely delivery and accurate decision-making.

The National Blood Bank Coordination Network addresses these challenges by creating a centralized platform that integrates blood

banks, hospitals, donors, transportation networks, and emergency response systems. The project demonstrates how Data Structures and Algorithms can be utilized to improve healthcare logistics and resource allocation.

2.2 CASE STUDY

The National Blood Bank Coordination Network is designed to simulate a real-world healthcare management system where multiple blood banks and hospitals operate within a coordinated national framework. The system focuses on maintaining blood inventories, managing donor records, processing emergency blood requests, optimizing transportation routes, and ensuring connectivity among healthcare facilities.

Hospitals continuously generate requests for different blood groups depending on patient requirements. These requests must be handled efficiently to prevent delays in treatment. The system uses a Queue data structure to ensure that requests are processed in the order they are received.

Blood inventories are maintained across multiple blood banks. To facilitate efficient allocation decisions, blood banks are ranked according to their available inventory using Merge Sort. This enables healthcare administrators to identify facilities with the highest stock levels.

Donor records are stored in sorted databases, allowing Binary Search to rapidly locate suitable donors. Transportation routes between blood banks and hospitals are represented as a graph structure, enabling Dijkstra's Algorithm to determine the shortest delivery routes. BFS is used to verify that all healthcare facilities remain connected within the transportation network.

The case study demonstrates how efficient use of Data Structures and Algorithms can significantly improve healthcare operations, reduce response times, and ensure better utilization of medical resources.

2.3 SCOPE OF THE PROBLEM

The scope of the National Blood Bank Coordination Network includes the management and coordination of blood-related healthcare operations across multiple facilities.

The system covers :-

- Blood Bank Inventory Management
- Donor Registration and Donor Search
- Hospital Database Management
- Emergency Blood Request Processing
- Blood Allocation and Distribution
- Transportation Route Optimization
- Network Connectivity Verification
- Inventory Ranking and Monitoring
- Healthcare Analytics and Reporting

The project focuses on demonstrating the practical application of Data Structures and Algorithms in solving healthcare logistics challenges. While the current implementation uses sample datasets, the architecture can be extended to support large-scale national healthcare systems with real-time databases and cloud-based infrastructure.

The scope also includes educational and academic learning objectives, helping students understand how theoretical DSA concepts can be applied to real-world healthcare scenarios.

OBJECTIVES OF THE PROJECT

The primary goal of the National Blood Bank Coordination Network is to improve the efficiency of blood resource management through the application of Data Structures and Algorithms.

3.1 To Develop a Centralized Blood Management System

To design and implement a unified platform that coordinates blood banks, hospitals, donors, and transportation facilities within a single healthcare network.

3.2 To Manage Blood Inventories Efficiently

To maintain accurate inventory records for different blood groups and provide real-time visibility into available stock levels across blood banks.

3.3 To Maintain Donor Registries

To store donor information, blood groups, contact details, and availability status in an organized and searchable format.

3.4 To Enable Fast Donor Search Operations

To implement Binary Search for rapid identification of suitable donors from sorted donor databases, reducing search time and improving operational efficiency.

3.5 To Process Emergency Blood Requests Systematically

To utilize Queue data structures for handling emergency blood requests in First-In-First-Out (FIFO) order, ensuring fair and organized request processing.

3.6 To Rank Blood Banks Based on Inventory Availability

To apply Merge Sort for arranging blood banks according to available inventory levels, enabling better allocation decisions during emergencies.

3.7 To Optimize Blood Distribution Routes

To use Dijkstra's Algorithm for identifying the shortest transportation routes between blood banks and hospitals, minimizing delivery time and improving emergency response.

3.8 To Verify Network Connectivity

To implement Breadth First Search (BFS) for checking connectivity among healthcare facilities and identifying isolated nodes within the transportation network.

3.9 To Generate Healthcare Analytics and Reports

To provide comprehensive reports containing inventory statistics, donor information, emergency request history, blood utilization trends, and operational insights that support informed decision-making.

SYSTEM OVERVIEW

The National Blood Bank Coordination Network is a healthcare logistics management system developed using C++ and Data Structures & Algorithms. The system is designed to coordinate blood banks, hospitals, donor registries, emergency response units, and transportation networks through a centralized platform.

The primary purpose of the system is to ensure efficient blood inventory management, rapid donor identification, organized emergency request processing, optimized transportation planning, and healthcare analytics reporting.

The system simulates a real-world national healthcare environment where multiple blood banks and hospitals work together to maintain adequate blood supplies and respond effectively to emergency situations.

The platform integrates several important Data Structures and Algorithms such as Queue, Binary Search, Merge Sort, Dijkstra's Algorithm, and Breadth First Search (BFS) to improve operational efficiency and decision-making.

The system follows a modular architecture where each component performs a specific healthcare management function while interacting with other modules to provide a complete blood coordination solution.

4.1 SYSTEM ARCHITECTURE

The National Blood Bank Coordination Network follows a centralized architecture model.

Architecture Flow

Donor Registration



Donor Database



Blood Inventory Management



Emergency Request Queue



Blood Allocation Engine



Transportation Network



Healthcare Analytics Dashboard

Main Components

1. Donor Registry

Stores donor details including :-

- Donor ID
- Name
- Blood Group
- Contact Information

- Availability Status

The donor registry enables quick donor identification during blood shortages.

2. Blood Bank Inventory System

Maintains inventory information for multiple blood banks.

Functions include:

- Blood Stock Monitoring
- Inventory Updates
- Blood Group Tracking
- Low Stock Detection
- Inventory Allocation

3. Hospital Database

Stores hospital records including :-

- Hospital ID
- Hospital Name
- City
- Emergency Blood Requirements

The hospital database acts as the request generation center within the system.

4. Emergency Response Queue

Handles emergency blood requests submitted by hospitals.

Queue follows:

FIFO (First In First Out)

The first request received is processed first, ensuring fairness and efficient emergency handling.

5. Inventory Ranking Engine

Ranks blood banks according to inventory availability.

Algorithm Used:

Merge Sort

Purpose:

- Faster allocation decisions
- Identification of highest inventory holders
- Efficient resource distribution

6. Donor Search Engine

Searches donor records efficiently.

Algorithm Used:

Binary Search

Purpose:

- Rapid donor identification

- Reduced search time
- Efficient emergency response

7. Route Optimization Module

Models transportation routes as weighted graphs.

Algorithm Used:

Dijkstra's Algorithm

Purpose:

- Find shortest delivery routes
- Minimize transportation time
- Improve emergency response speed

8. Network Connectivity Module

Verifies connectivity among healthcare facilities.

Algorithm Used:

Breadth First Search (BFS)

Purpose:

- Identify isolated facilities
- Ensure network reliability
- Validate transportation accessibility

9. Healthcare Analytics Dashboard

Generates reports containing :-

- Total Blood Units
- Available Donors
- Request History
- Blood Bank Statistics
- Emergency Request Analytics
- Most Requested Blood Group

4.2 LAYER DISTRIBUTION

The system follows a layered architecture model that separates responsibilities into multiple logical layers.

Layer 1:- User Interaction Layer

This is the top-most layer of the system.

Responsibilities:

- Accept user input
- Display menus
- Show inventory reports
- Display donor information
- Present analytics dashboards

Examples :-

Display Donors

Display Hospitals

Show Inventory

Generate Reports

Emergency Queue Dashboard

Layer 2 :- Business Logic Layer

This layer contains the core functionality of the application.

Responsibilities :-

- Process requests
- Validate data
- Allocate blood units
- Manage donor records
- Perform inventory calculations

Examples :-

Add Donor

Add Inventory

Process Requests

Generate Analytics

Request History Management

Layer 3 :- Algorithm Layer

This layer contains all Data Structures and Algorithms.

Responsibilities :-

- Sorting
- Searching
- Queue Management
- Route Optimization
- Connectivity Verification

Implemented Algorithms:

Queue

Binary Search

Merge Sort

Dijkstra's Algorithm

BFS

This is the most important layer of the project because it demonstrates practical DSA implementation.

Layer 4 :- Data Management Layer

Responsible for storing and organizing information.

Data Structures Used :-

Vector

Map

Queue

Graph

Stored Information :-

- Donors
- Hospitals
- Blood Banks
- Inventories
- Requests
- Transportation Network

Layer 5 :- Reporting Layer

Responsible for generating analytical information.

Functions :-

- Blood Inventory Reports
- Donor Statistics
- Emergency Request Reports
- Healthcare Analytics
- Operational Summaries

The reporting layer converts raw data into meaningful insights for healthcare administrators.

SYSTEM DESIGN AND ARCHITECTURE

The National Blood Bank Coordination Network is designed using Object-Oriented Programming principles and modular software architecture. Each module performs a specific function and collaborates with other modules to achieve overall system objectives.

The architecture focuses on scalability, maintainability, efficiency, and real-world applicability.

5.1 DESIGN PRINCIPLES

The project follows the following software design principles :-

Modularity

Each component performs a separate function.

Examples :-

- Donor Module
- Hospital Module
- Blood Bank Module
- Request Module
- Network Module

Reusability

Classes and functions are designed for reuse throughout the application.

Examples :-

Binary Search Function

Merge Sort Function

Inventory Functions

Route Optimization Functions

Scalability

The architecture can easily accommodate :-

- Additional hospitals
- Additional blood banks
- Additional donors
- Larger transportation networks

Maintainability

The modular structure simplifies debugging, testing, and future enhancements.

5.2 HIGH LEVEL DESIGN

System Components :-

User



Menu Driven Interface



Blood Management Modules



DSA Algorithms



Reports & Analytics



Output Dashboard

The High-Level Design demonstrates the interaction between users, modules, algorithms, and reports.

5.3 LOW LEVEL DESIGN

Donor Class

Stores :-

- ID
- Name
- Blood Group
- Phone Number
- Availability Status

Blood Bank Class

Stores :-

- Blood Bank ID
- Name
- City
- Inventory Records

Hospital Class

Stores:

- Hospital ID
- Name
- City

Request Class

Stores:

- Hospital Name
- Blood Group
- Required Units
- Timestamp

Network Class

Stores:

- Transportation Graph
- Nodes

- Edges
- Route Information

Provides:

- Dijkstra Algorithm
- BFS Connectivity Check

5.4 SYSTEM WORKFLOW

Step 1

Initialize blood banks, hospitals, donors, and transportation routes.



Step 2

User selects an operation from the menu.



Step 3

System processes the request.



Step 4

Appropriate DSA algorithm executes.



Step 5

Results are displayed.



Step 6

Reports are generated.



Step 7

System waits for next operation.

5.5 ARCHITECTURAL BENEFITS

The architecture provides :-

- Fast donor search operations
- Efficient inventory ranking
- Organized emergency request processing
- Optimal route calculation
- Reliable network verification
- Detailed healthcare analytics
- Improved maintainability
- Better scalability
- Real-world healthcare applicability

The design demonstrates how Data Structures and Algorithms can be integrated into a practical healthcare logistics management system to improve operational efficiency and support life-saving medical operations.

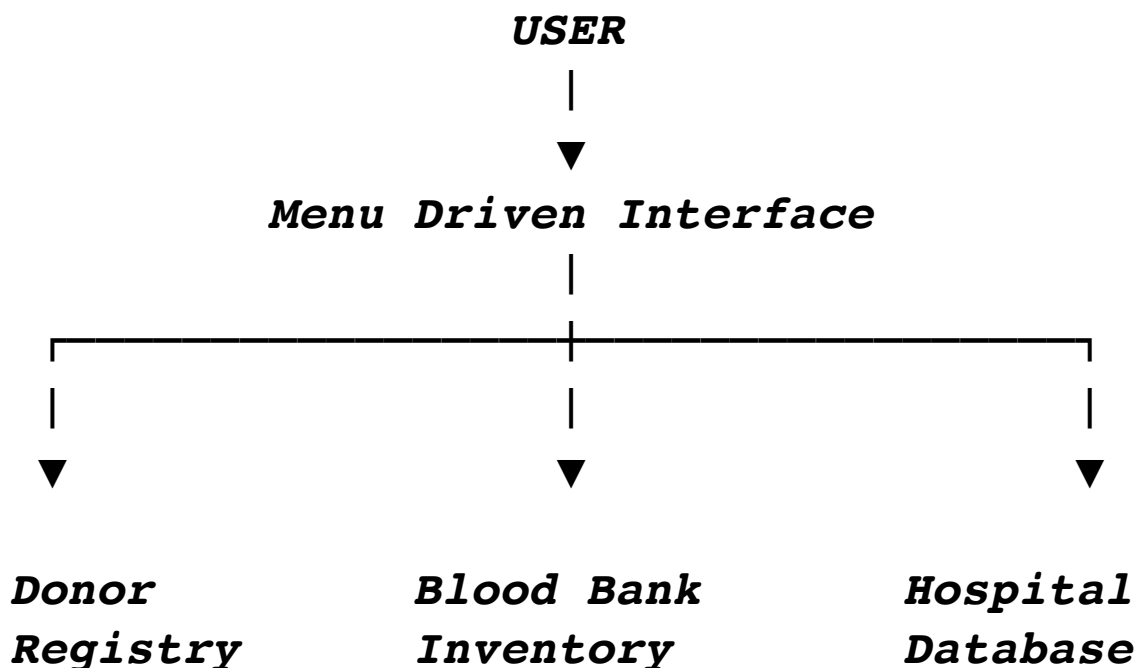
MODULE INTERACTION

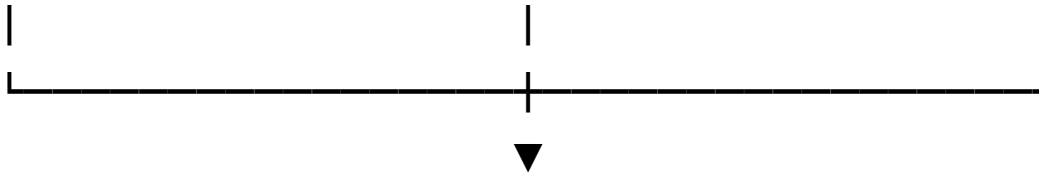
6.1 INTRODUCTION

The National Blood Bank Coordination Network consists of multiple interconnected modules that work together to provide efficient blood inventory management, donor coordination, emergency request processing, route optimization, and healthcare analytics. Each module performs a specific task while continuously exchanging information with other modules to ensure smooth system operation.

The interaction among these modules enables real-time decision-making and efficient healthcare logistics management. The modular architecture improves maintainability, scalability, and operational efficiency while demonstrating the practical implementation of Data Structures and Algorithms.

6.2 MODULE INTERACTION DIAGRAM





Emergency Request Queue



Blood Allocation Engine



***Inventory Ranking Module
(Merge Sort)***



***Donor Search Module
(Binary Search)***



***Route Optimization Module
(Dijkstra Algorithm)***



***Network Connectivity Module
(BFS)***



Analytics & Reporting System



Final Output

6.3 DONOR REGISTRY INTERACTION

The Donor Registry module stores and manages donor information including donor ID, name, blood group, contact details, and availability status.

Interactions

- Receives donor registration information from users.
- Sends donor records to the Binary Search module.
- Provides donor details during blood shortages.
- Supports emergency response operations.

Data Exchanged

Input :-

- Donor ID
- Name
- Blood Group
- Phone Number

Output :-

- Donor Information

- Available Donor List

6.4 BLOOD BANK INVENTORY INTERACTION

The Blood Bank Inventory module maintains blood stock information for all blood banks within the network.

Interactions

- Receives inventory updates from administrators.
- Supplies stock information to the ranking module.
- Supports emergency request processing.
- Generates inventory reports.

Data Exchanged

Input :-

- Blood Group
- Number of Units

Output :-

- Available Inventory
- Low Stock Alerts
- Total Inventory Statistics

6.5 HOSPITAL DATABASE INTERACTION

The Hospital Database module stores information about hospitals connected to the network.

Interactions

- Generates emergency blood requests.
- Provides destination information for route optimization.
- Supports healthcare analytics.

Data Exchanged

Input :-

- Hospital ID
- Hospital Name
- City

Output :-

- Hospital Records
- Emergency Request Details

6.6 EMERGENCY REQUEST QUEUE INTERACTION

The Emergency Request Queue is one of the most critical modules in the system.

Algorithm Used

Queue (FIFO)

Interactions

- Receives requests from hospitals.

- Stores requests in FIFO order.
- Sends requests to the allocation engine.
- Updates request history records.

Process Flow

Hospital Request

↓

Emergency Queue

↓

Request Processing

↓

Blood Allocation

↓

History Storage

Benefits

- Fair request handling.
- Organized emergency management.
- Reduced processing conflicts.

6.7 INVENTORY RANKING MODULE INTERACTION

The Inventory Ranking Module ranks blood banks according to available stock.

Algorithm Used

Merge Sort

Interactions

- Receives inventory data from blood banks.
- Sorts blood banks by stock availability.
- Provides ranked information to allocation systems.

Example

Before Ranking

Blood Bank A = 60 Units

Blood Bank B = 100 Units

Blood Bank C = 30 Units

After Ranking

Blood Bank B = 100 Units

Blood Bank A = 60 Units

Blood Bank C = 30 Units

Benefits

- Faster allocation decisions.
- Better resource utilization.
- Improved emergency response.

6.8 DONOR SEARCH MODULE INTERACTION

The Donor Search Module helps identify suitable donors quickly.

Algorithm Used

Binary Search

Interactions

- Receives donor ID from users.
- Searches sorted donor database.
- Returns matching donor information.

Example

Search Donor ID = 102

Output:

Priya | A+ | 9876543211

Benefits

- $O(\log n)$ search complexity.
- Rapid donor identification.
- Improved healthcare response time.

6.9 ROUTE OPTIMIZATION MODULE INTERACTION

This module determines the shortest transportation path between blood banks and hospitals.

Algorithm Used

Dijkstra's Algorithm

Interactions

- Receives source and destination nodes.
- Uses transportation network graph.
- Calculates shortest delivery path.

Example

Source :-

City Blood Bank

Destination:

Grace Clinic

Output :-

City Blood Bank → LifeCare Bank → Grace Clinic

Distance = 18 km

Benefits

- Faster blood delivery.
- Reduced transportation cost.
- Improved emergency response.

6.10 NETWORK CONNECTIVITY MODULE INTERACTION

This module verifies whether all healthcare facilities remain connected.

Algorithm Used

Breadth First Search (BFS)

Interactions

- Receives graph data from network module.
- Traverses connected facilities.
- Detects isolated nodes.

Example Output

Network Status :-

ALL FACILITIES CONNECTED

Benefits

- Network reliability.
- Early fault detection.
- Improved operational stability.

6.11 ANALYTICS AND REPORTING MODULE INTERACTION

The Analytics Module collects information from all modules and generates reports.

Information Collected

- Donor Statistics

- Blood Inventory Data
- Emergency Requests
- Completed Requests
- Blood Utilization Trends
- Network Information

Generated Reports

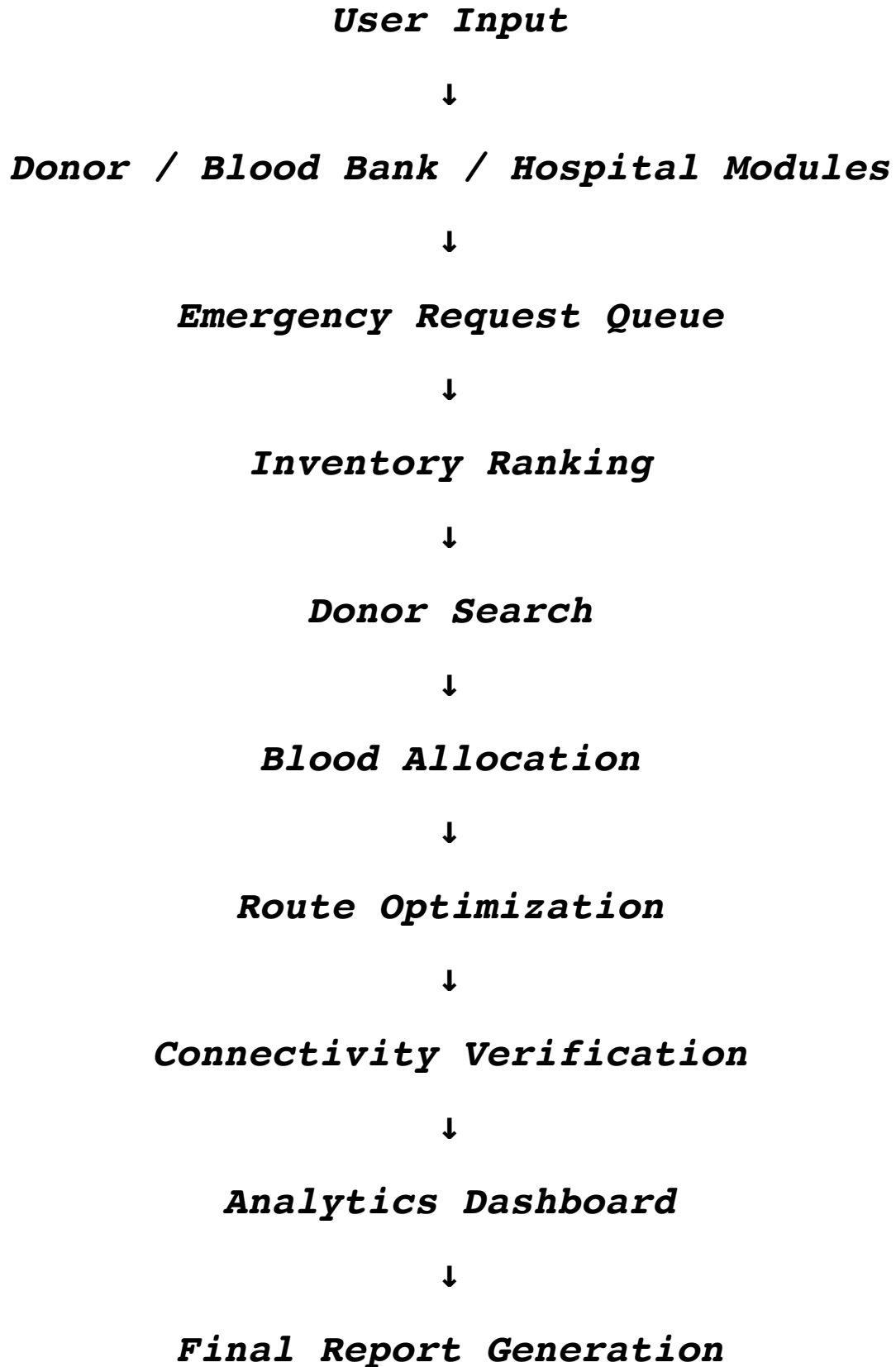
- Total Blood Banks
- Total Donors
- Available Donors
- Pending Requests
- Completed Requests
- Total Blood Units
- Most Requested Blood Group

Benefits

- Better decision-making.
- Performance monitoring.
- Resource planning.

6.12 OVERALL MODULE COMMUNICATION FLOW

The complete interaction flow of the National Blood Bank Coordination Network can be summarized as :-



This interaction mechanism ensures that all healthcare operations are coordinated efficiently while utilizing Data Structures and Algorithms to achieve optimal performance and reliable healthcare logistics management.

DATA STRUCTURES AND ALGORITHMS USED

The National Blood Bank Coordination Network is designed using multiple Data Structures and Algorithms to efficiently manage healthcare logistics, blood inventory allocation, donor searching, emergency request processing, and transportation route optimization. Each data structure and algorithm has been selected according to its suitability for solving a specific real-world problem described in the case study.

7.1 Vector (Dynamic Array)

Purpose

Vectors are used to store and manage collections of :-

- Blood Banks
- Donors
- Hospitals
- Completed Emergency Requests

The C++ STL Vector provides dynamic memory allocation and allows records to grow automatically as new entries are added.

Why Vector ?

Healthcare systems continuously receive new donor registrations, hospital records, and blood inventory updates. Therefore, a dynamic storage structure is required.

Advantages :-

- Dynamic Size
- Fast Random Access
- Easy Traversal
- Efficient Memory Management

Implementation in Project

```
vector<BloodBank> bloodBanks
```

```
vector<Donor> donors
```

```
vector<Hospital> hospitals
```

```
vector<Request> completedRequests
```

```
vector<BloodBank> bloodBanks
```

Operations Performed

- Add Donor
- Store Blood Banks
- Store Hospitals
- Maintain Request History
- Generate Reports

Time Complexity

Operation	Complexity
Access	$O(1)$
Insert at End	$O(1)$
Search	$O(n)$

7.2 Queue

Purpose

The Queue Data Structure is used for Emergency Request Processing.

Hospitals continuously submit blood requests during emergencies. These requests must be processed in the exact order in which they are received.

The First-In-First-Out (FIFO) principle makes Queue the ideal structure for this task.

Why Queue ?

Consider the following scenario :-

Hospital A requests blood at 10:00 AM.

Hospital B requests blood at 10:05 AM.

Hospital C requests blood at 10:10 AM.

The system must serve Hospital A first, followed by B and then C.

Queue naturally follows this sequence.

Implementation

Table 3

```
queue<Request> emergencyQueue;
```

Queue Operations Used

Push Operation

```
emergencyQueue.push(Request(...));
```

Adds a new emergency request.

Pop Operation

```
emergencyQueue.pop();
```

Removes the processed request.

Front Operation

```
emergencyQueue.front();
```

Accesses the oldest request.

Real-Life Importance

In healthcare systems, fairness and urgency are critical. Queue ensures that no request is skipped and all hospitals are served systematically.

Time Complexity

Operation	Complexity
Push	$O(1)$
Pop	$O(1)$
Front	$O(1)$

7.3 Map

Purpose

Maps are used to store Blood Group Inventory information.

Each blood bank contains multiple blood groups and corresponding stock quantities.

Why Map ?

A blood bank inventory can be represented as :-

Blood Group	Units
O+	50
A+	100
O-	10

Searching by blood group should be fast and organized.

Map provides :-

- Key-Value Storage
- Efficient Lookup
- Automatic Sorting of Keys

Implementation

```
map<string, int> inventory;
```

Example :-

inventory["O+"]	=	50;
inventory["A+"]	=	100;

Operations

- Add Inventory
- Update Inventory
- Search Blood Group Stock
- Generate Reports

Time Complexity

Operation	Complexity
Insert	$O(\log n)$
Search	$O(\log n)$
Update	$O(\log n)$

7.4 Binary Search Algorithm

Purpose

Binary Search is used to locate donors quickly using their unique Donor ID.

Why Binary Search ?

In large healthcare systems there may be thousands of registered donors.

Searching each donor one by one would be inefficient.

Binary Search significantly reduces search time by repeatedly dividing the search space into halves.

Working Principle

Suppose donor IDs are :-

101 102 103 104 105

Searching for :-

102

Steps :-

1. Check middle element.
2. Compare with target.
3. Move left or right.
4. Repeat until found.

Implementation

```
int binarySearchDonor(const vector<Donor>& donors, int targetId)
```

Advantages

- Fast Searching
- Suitable for Sorted Data
- Efficient for Large Databases

Time Complexity

Case	Complexity
Best	$O(1)$
Average	$O(\log n)$
Worst	$O(\log n)$

7.5 Merge Sort Algorithm

Purpose

Merge Sort is used to rank blood banks according to total available inventory.

Why Merge Sort ?

When an emergency occurs, the healthcare authority must identify which blood bank has the highest stock.

Sorting helps prioritize blood allocation.

Merge Sort is chosen because :-

- Highly Efficient
- Stable Sorting Algorithm
- Guaranteed $O(n \log n)$ Performance

Working Principle

Divide

Split the list into smaller halves.

Conquer

Sort each half recursively.

Merge

Combine sorted halves into a final sorted list.

Example

Before Sorting

Blood Bank	Inventory Units
Bank A	60 Units
Bank B	100 Units
Bank C	30 Units

After Sorting

Rank	Blood Bank	Inventory Units
1	Bank B	100 Units
2	Bank A	60 Units

3	Bank C	30 Units
---	--------	----------

Implementation

```
mergeSort(bloodBanks, 0, bloodBanks.size()-1);
```

Benefits

- Efficient Ranking
- Supports Large Datasets
- Stable Ordering

Time Complexity

Case	Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$

7.6 Graph Data Structure

Purpose

The transportation network between blood banks and hospitals is represented using a Graph.

Why Graph ?

Healthcare facilities are connected through roads and transportation routes.

Each facility becomes a node.

Each road becomes an edge.

Graph Representation :-

City Blood Bank

/

/

LifeCare Bank

/

/

Grace Clinic

Project Nodes

Node ID	Facility
0	City Blood Bank
1	LifeCare Bank
2	RedCross Bank
3	City Hospital
4	Grace Clinic

Transportation Routes

Source Node	Destination Node	Distance
City Blood Bank	City Hospital	5 km
LifeCare Bank	Grace Clinic	3 km
City Blood Bank	LifeCare Bank	15 km
RedCross Bank	City Hospital	2 km
City Blood Bank	RedCross Bank	4 km

Graph Edges

cityNetwork->addEdge(0	3	5);
cityNetwork->addEdge(1	4	3);
cityNetwork->addEdge(0	1	15);
cityNetwork->addEdge(2	3	2);
cityNetwork->addEdge(0	2	4);

Benefits

- Represents Real Road Networks
- Supports Route Optimization
- Supports Connectivity Analysis

7.7 Dijkstra's Algorithm

Purpose

Dijkstra's Algorithm is used to find the shortest route between healthcare facilities.

Why Dijkstra ?

In emergency situations every minute matters. Blood should be delivered through the shortest available route.

Sample Route Optimisation :-

Source :-

City Blood Bank

Destination :-

Grace Clinic

Output :-

Shortest Distance :- 18 km

Path :-

City Blood Bank

→ LifeCare Bank

→ Grace Clinic

Benefits

- Fast Route Calculation
- Reduced Transportation Time
- Supports Emergency Logistics

Time Complexity Table

Complexity
$O((V + E) \log V)$

Parameters

Symbol	Meaning
V	Number of Vertices (Facilities)
E	Number of Edges (Routes)

7.8 Breadth First Search (BFS)

Purpose

BFS is used to verify whether the healthcare network remains connected.

Why BFS ?

If a blood bank becomes isolated, emergency deliveries may fail.

BFS checks whether every facility can be reached from every other facility.

Working Principle

- 1. Start from a node.
- 2. Visit all neighboring nodes.
- 3. Continue level by level.
- 4. Verify connectivity.

Connectivity Verification Result

Network Status
ALL FACILITIES CONNECTED

Benefits

- Detects Isolated Facilities
- Improves Reliability
- Ensures National Coverage

Time Complexity Table

Complexity
$O(V + E)$

7.9 Summary of Data Structures and Algorithms

Data Structure / Algorithm	Purpose
Vector	Store donors, hospitals, blood banks
Queue	Process emergency requests
Map	Manage blood inventory
Binary Search	Search donor by ID
Merge Sort	Rank blood banks
Graph	Model transportation network
Dijkstra's Algorithm	Find shortest delivery route
BFS	Verify network connectivity

EXTRA FEATURES IMPLEMENTED

The National Blood Bank Coordination Network includes several additional features beyond the core requirements specified in the case study. These features improve usability, reliability, healthcare analytics, and operational efficiency.

9.1 Donor Availability Tracking

The system maintains donor availability status to ensure that only eligible and available donors are displayed during blood searches.

Benefits

- Prevents contacting unavailable donors.
- Improves emergency response efficiency.
- Maintains an updated donor registry.

9.2 Blood Group Validation

The system validates blood group inputs before storing donor records or processing requests.

Supported Blood Groups

Blood Group
O+
O-
A+
A-
B+
B-
AB+
AB-

Benefits

- Eliminates invalid entries.
- Maintains database consistency.
- Improves data reliability.

9.3 Duplicate Donor Prevention

The system checks donor IDs and phone numbers before registration.

Benefits

- Prevents duplicate records.
- Improves database accuracy.
- Ensures unique donor identification.

9.4 Low Stock Alert System

The inventory dashboard automatically generates alerts when blood stock falls below a predefined threshold.

Benefits

- Prevents inventory shortages.
- Supports proactive inventory management.
- Enhances emergency preparedness.

9.5 Emergency Response Dashboard

A dedicated dashboard displays all pending emergency blood requests.

Benefits

- Provides real-time visibility.
- Improves emergency coordination.
- Supports faster request processing.

9.6 Request History Tracking

All completed emergency requests are stored along with timestamps.

Benefits

- Enables auditing.

- Supports healthcare analytics.
- Maintains historical records.

9.7 Healthcare Analytics Report

The system generates a detailed operational report containing :-

- Total Blood Banks
- Total Donors
- Available Donors
- Completed Requests
- Pending Requests
- Total Blood Units
- Low Stock Alerts
- Most Requested Blood Group

Benefits

- Supports decision-making.
- Improves resource planning.
- Enables healthcare analysis.

IMPLEMENTATION APPROACH

The project follows a modular and object-oriented implementation approach. Each component is designed independently while interacting with other modules to provide complete healthcare logistics management.

10.1 PROJECT FOLDER STRUCTURE

The project is organized as follows :-

DSA-FINAL-PROJECT

```
|
|
|— main.cpp
|
|
|— sample_input.txt
|
|
|— sample_output.txt
|
|
|— README.md
|
|
|— bloodbank
|
|
└— blood_bank
```

Description

File Name	Purpose
main.cpp	Complete source code
sample_input.txt	Sample test inputs
sample_output.txt	Sample execution outputs
README.md	Project documentation
bloodbank	Compiled executable
blood_bank	Additional executable file

10.2 CLASS DESIGN

The project follows Object-Oriented Programming principles using multiple classes.

Donor Class

Stores donor information.

Attributes

Attribute	Description
id	Unique donor ID
name	Donor name
bloodGroup	Blood group
phone	Contact number
available	Availability status

Responsibilities

- Store donor information.
- Support donor searching.
- Manage donor availability.

BloodBank Class

Stores blood bank information and inventory.

Attributes

Attribute	Description
id	Blood bank ID
name	Blood bank name
city	Location
inventory	Blood inventory map

Responsibilities

- Manage blood stock.
- Calculate total inventory.
- Support inventory ranking.

Hospital Class

Stores hospital information.

Attributes

Attribute	Description
id	Hospital ID
name	Hospital name
city	Hospital location

Responsibilities

- Store hospital details.
- Support emergency request generation.

Request Class

Stores emergency blood request information.

Attributes

Attribute	Description
hospitalName	Requesting hospital
bloodGroup	Required blood group
units	Required units
timestamp	Request time

Responsibilities

- Store emergency request data.
- Maintain request history.

Network Class

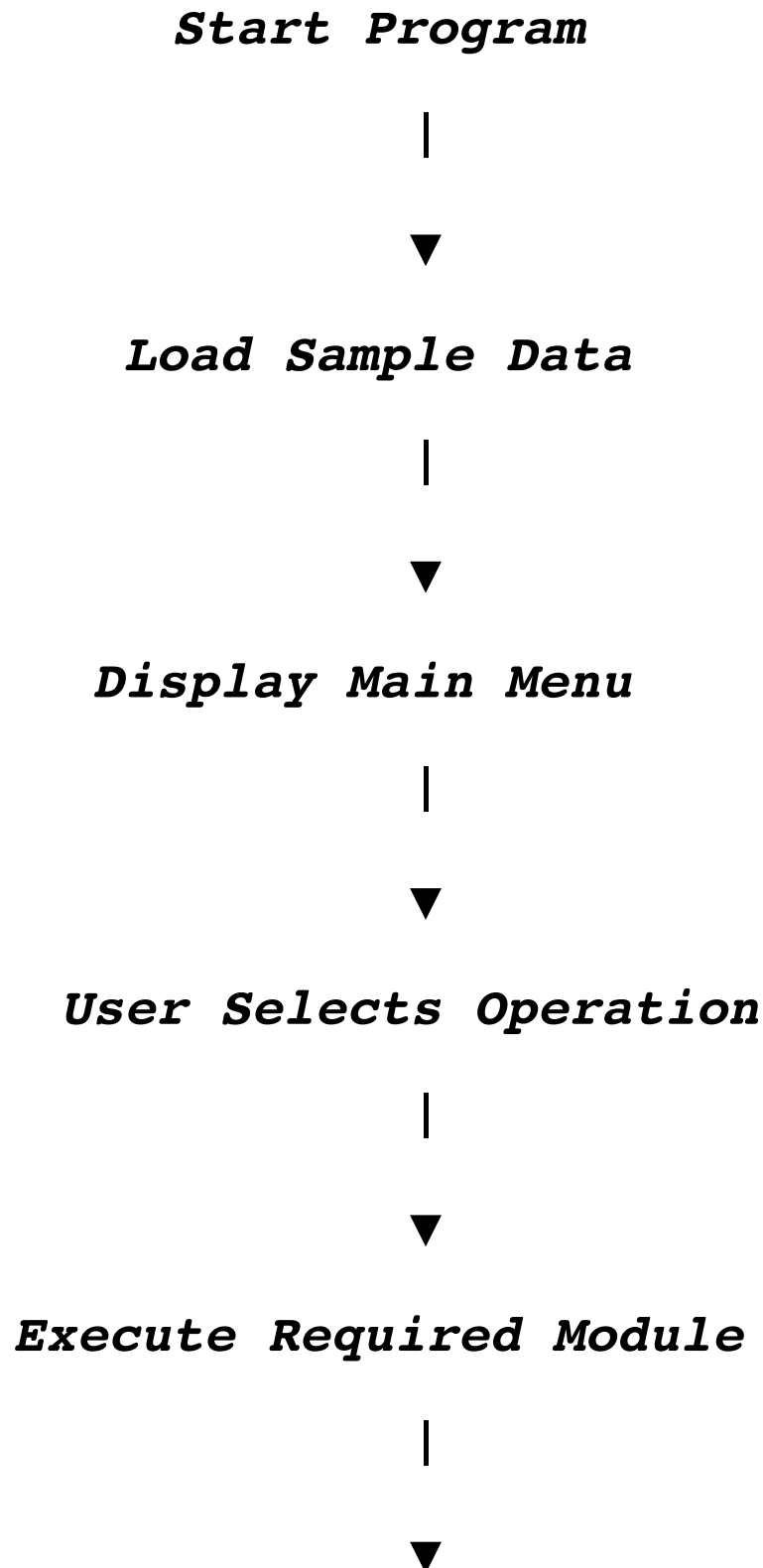
Represents transportation routes using a graph.

Responsibilities

- Store network connections.
- Execute Dijkstra's Algorithm.
- Execute BFS Connectivity Check.
- Display transportation routes.

10.3 PROGRAM WORKFLOW

The overall workflow of the system is shown below.



Update Records



Display Results



Generate Reports



Exit Program

Detailed Workflow

Step 1

Initialize blood banks, donors, hospitals, and transportation network.

Step 2

Display menu-driven interface.

Step 3

Accept user input.

Step 4

Execute requested operation :-

- Donor Management
- Inventory Management
- Blood Search
- Emergency Request Processing
- Route Optimization
- Connectivity Verification
- Report Generation

Step 5

Display results.

Step 6

Continue until user selects Exit.

COMPLEXITY ANALYSIS

Complexity analysis helps evaluate the performance and scalability of the system.

11.1 DATA STRUCTURE COMPLEXITY

Data Structure	Operation	Time Complexity
Vector	Access	$O(1)$
Vector	Search	$O(n)$
Vector	Insert End	$O(1)$
Queue	Push	$O(1)$
Queue	Pop	$O(1)$
Queue	Front	$O(1)$
Map	Insert	$O(\log n)$
Map	Search	$O(\log n)$
Map	Delete	$O(\log n)$

Observations

- Queue operations execute in constant time.
- Maps efficiently manage blood inventories.
- Vectors provide fast storage and retrieval.

11.2 ALGORITHM COMPLEXITY

Algorithm	Purpose	Time Complexity
Binary Search	Donor Search	$O(\log n)$
Merge Sort	Inventory Ranking	$O(n \log n)$
Dijkstra's Algorithm	Route Optimization	$O((V+E) \log V)$

BFS	Connectivity Verification	$O(V+E)$
-----	---------------------------	----------

Algorithm Summary

Algorithm	Best Case	Average Case	Worst Case
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
BFS	$O(V+E)$	$O(V+E)$	$O(V+E)$
Dijkstra	$O((V+E) \log V)$	$O((V+E) \log V)$	$O((V+E) \log V)$

SAMPLE DATASET

The National Blood Bank Coordination Network is initialized with sample blood banks, hospitals, donors, blood inventories, and transportation routes to simulate real-world healthcare logistics operations. The dataset represents a centralized healthcare ecosystem where blood banks maintain inventories, hospitals generate emergency blood requests, and registered donors provide additional support during shortages. Transportation routes between healthcare facilities are modeled as a network graph, enabling route optimization and efficient blood distribution across different locations.

12.1 Blood Bank Dataset

Blood Bank ID	Blood Bank Name	City	O+ Units	O- Units	A+ Units	Total Inventory
0	City Blood Bank	Mumbai	50	10	0	60
1	LifeCare Bank	Pune	0	0	100	100
2	RedCross Bank	Mumbai	30	0	0	30

12.2 Donor Dataset

Donor ID	Donor Name	Blood Group	Phone Number	Availability
101	Rahul	O+	9876543210	Available
102	Priya	A+	9876543211	Available
103	Aman	B-	9876543212	Available
104	Simran	O-	9876543213	Available
105	Karan	AB+	9876543214	Available

12.3 Hospital Dataset

Hospital ID	Hospital Name	City
3	City Hospital	Mumbai
4	Grace Clinic	Pune

12.4 Transportation Network Dataset

Source Node	Destination Node	Distance (km)
City Blood Bank	City Hospital	5
LifeCare Bank	Grace Clinic	3
City Blood Bank	LifeCare Bank	15
RedCross Bank	City Hospital	2
City Blood Bank	RedCross Bank	4

SAMPLE INPUTS AND OUTPUTS

Sample Input

15
2
3
102
4
O+
6
7
17
10
0
4
11
8
City Hospital
O+
20
16
9
12
13
14

Sample Output 1 – Hospital Database

--- Hospital Database ---

ID	Hospital Name	City
3	City Hospital	Mumbai
4	Grace Clinic	Pune

Sample Output 2 – Donor Search

Enter Donor ID :- 102

Donor Found :-

Priya | Group: A+ | Phone: 9876543211

Sample Output 3 – Blood Group Search

Available Donors for O+

Rahul (ID :- 101)

Sample Output 4 – Inventory Ranking

Blood Banks Ranked by Total Inventory

LifeCare Bank – 100 Units

City Blood Bank – 60 Units

RedCross Bank – 30 Units

Sample Output 5 – Route Optimization

Shortest Distance :- 18 km

Path :-

City Blood Bank → LifeCare Bank → Grace Clinic

Sample Output 6 – Connectivity Verification

Network Status :-

ALL FACILITIES CONNECTED

Sample Output 7 – Emergency Request Processing

Processing Request for City Hospital

Allocated from City Blood Bank

Remaining Stock :- 30 Units

Sample Output 8 – Healthcare Analytics Report

Total Blood Banks :- 3

Total Donors :- 5

Available Donors :- 5

Completed Requests :- 1

Pending Requests :- 0

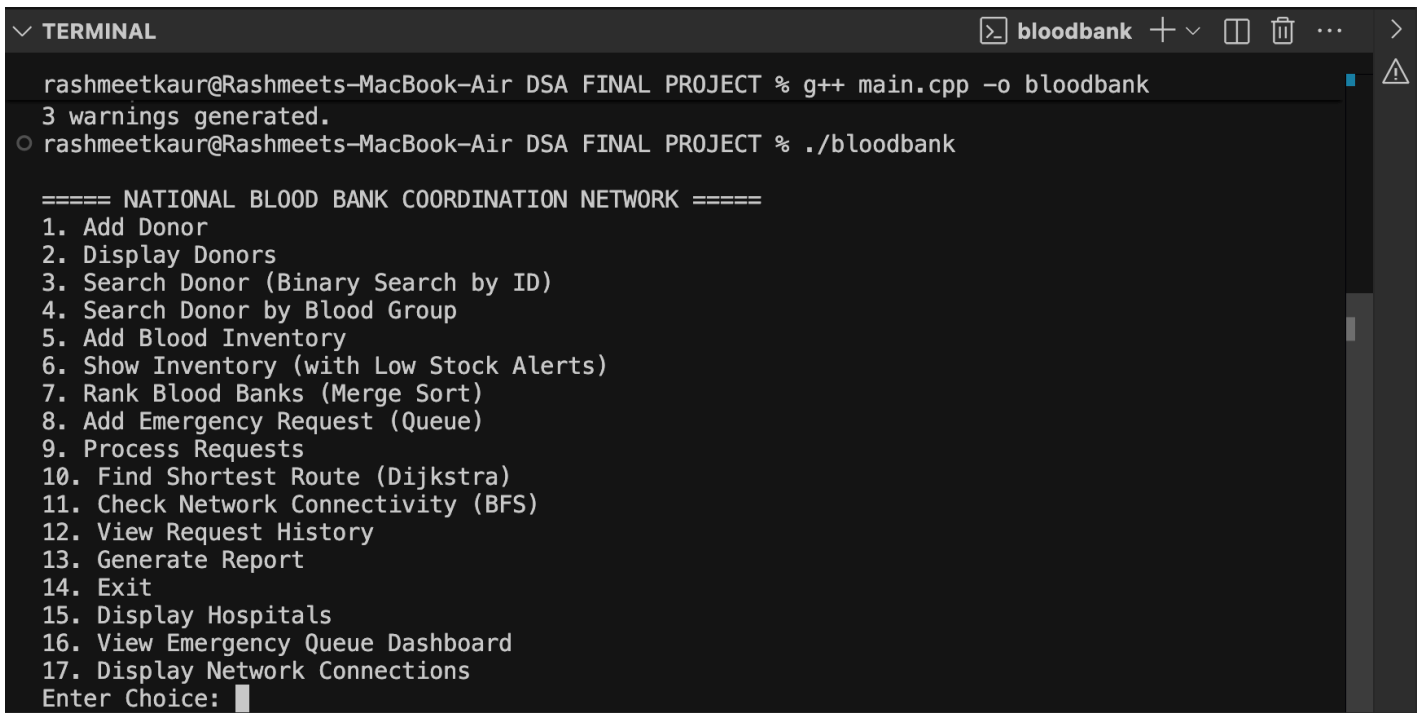
Most Requested Blood Group :- O+

SCREENSHOTS OF EXECUTION

The following screenshots demonstrate the successful execution of the National Blood Bank Coordination Network system.

Screenshots were captured during project execution and illustrate the implementation of Data Structures and Algorithms including Queue, Binary Search, Merge Sort, Dijkstra's Algorithm, and BFS.

Figure 1:- Main Menu Interface



```
▼ TERMINAL bloodbank + - [] [x] ... >
rashmeetkaur@Rashmeets-MacBook-Air DSA FINAL PROJECT % g++ main.cpp -o bloodbank
3 warnings generated.
○ rashmeetkaur@Rashmeets-MacBook-Air DSA FINAL PROJECT % ./bloodbank

===== NATIONAL BLOOD BANK COORDINATION NETWORK =====
1. Add Donor
2. Display Donors
3. Search Donor (Binary Search by ID)
4. Search Donor by Blood Group
5. Add Blood Inventory
6. Show Inventory (with Low Stock Alerts)
7. Rank Blood Banks (Merge Sort)
8. Add Emergency Request (Queue)
9. Process Requests
10. Find Shortest Route (Dijkstra)
11. Check Network Connectivity (BFS)
12. View Request History
13. Generate Report
14. Exit
15. Display Hospitals
16. View Emergency Queue Dashboard
17. Display Network Connections
Enter Choice: █
```

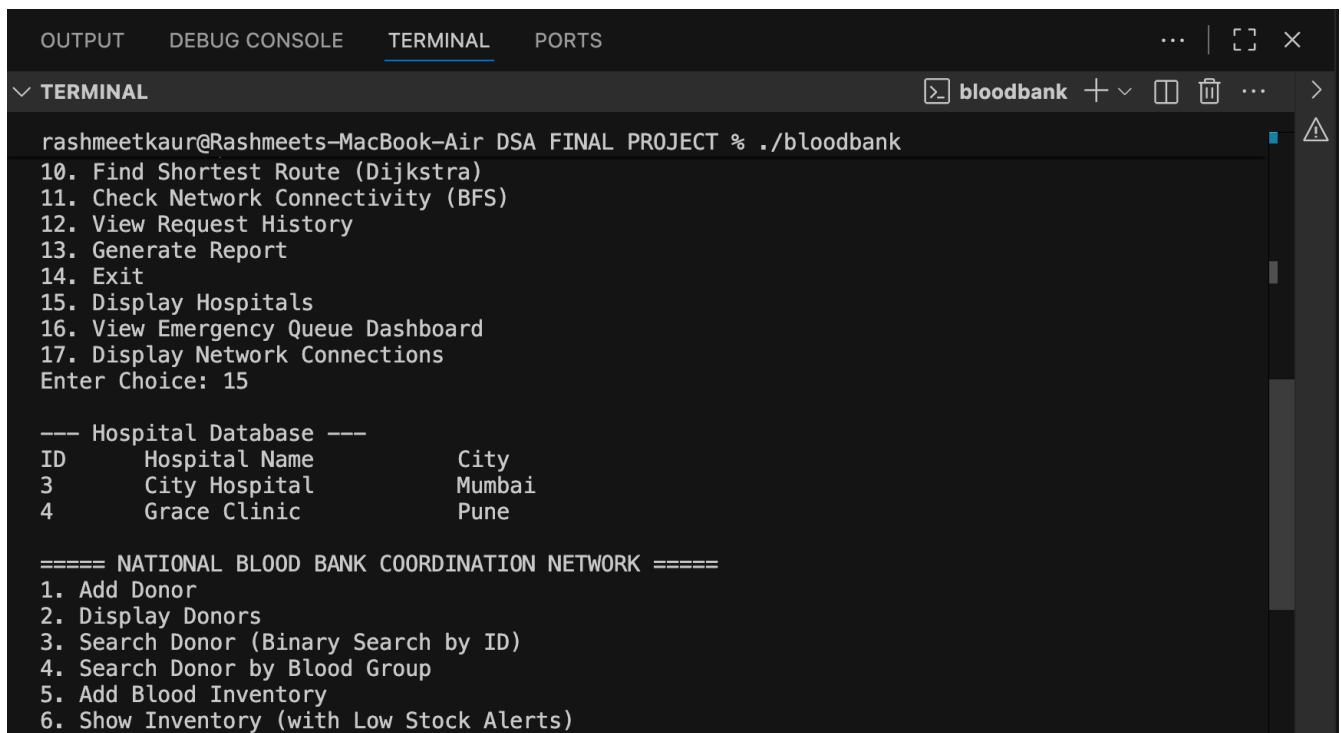
Description

The Main Menu Interface acts as the central control panel of the National Blood Bank Coordination Network. It provides access to all healthcare management modules including donor management, blood inventory monitoring, emergency request processing, blood bank ranking, route optimization, network connectivity verification, and healthcare analytics reporting. The menu-driven architecture improves usability by allowing users to navigate different functionalities through numbered options.

Data Structures and Algorithms Demonstrated :-

- Queue
- Binary Search
- Merge Sort
- Graph
- Dijkstra's Algorithm
- Breadth First Search (BFS)

Figure 2 :- Hospital Database



The screenshot shows a terminal window with the following content:

```
OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
✓ TERMINAL  bloodbank + - [ ] [ ] ... >
rashmeetkaur@Rashmeets-MacBook-Air DSA FINAL PROJECT % ./bloodbank
10. Find Shortest Route (Dijkstra)
11. Check Network Connectivity (BFS)
12. View Request History
13. Generate Report
14. Exit
15. Display Hospitals
16. View Emergency Queue Dashboard
17. Display Network Connections
Enter Choice: 15

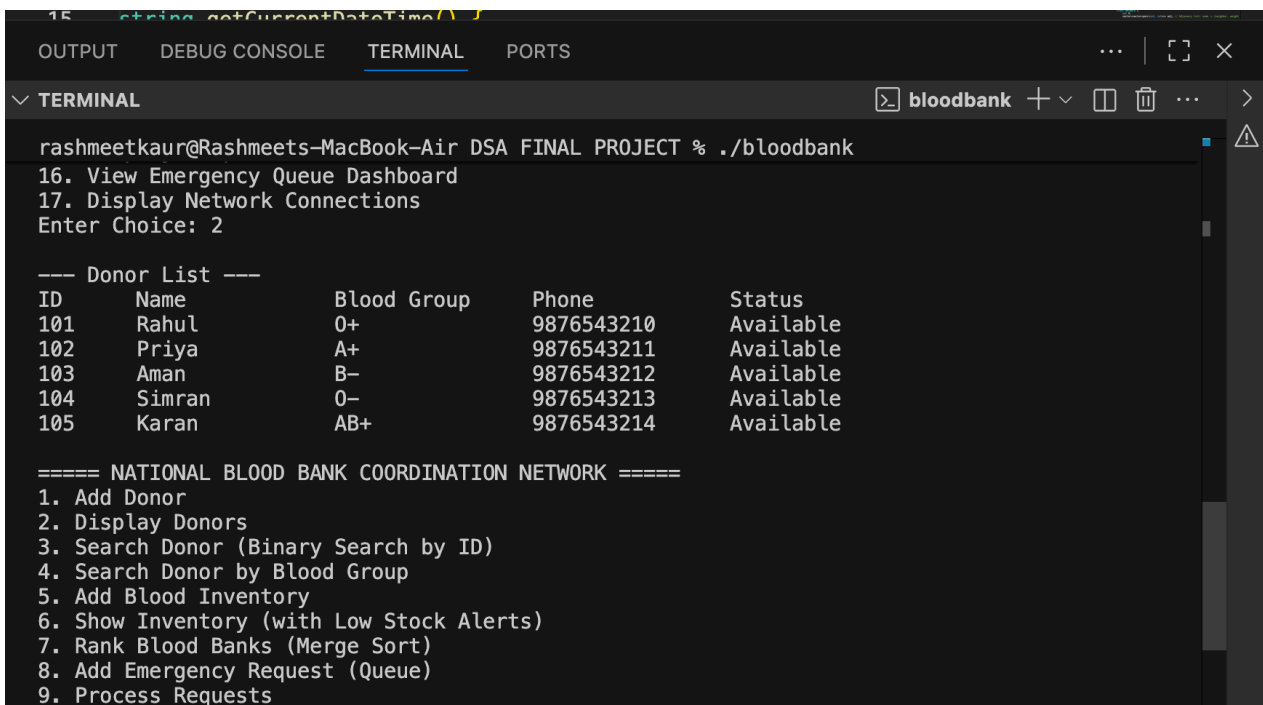
--- Hospital Database ---
ID      Hospital Name      City
3       City Hospital      Mumbai
4       Grace Clinic          Pune

===== NATIONAL BLOOD BANK COORDINATION NETWORK =====
1. Add Donor
2. Display Donors
3. Search Donor (Binary Search by ID)
4. Search Donor by Blood Group
5. Add Blood Inventory
6. Show Inventory (with Low Stock Alerts)
```

Description

The Hospital Database module displays all hospitals registered within the healthcare network. Each hospital record contains a unique identifier, hospital name, and location. This module helps maintain a centralized repository of healthcare facilities that can request blood units during emergencies. The information is used by the route optimization and emergency response modules.

Figure 3 :- Donor Registry



```
15 string getCurrentDateTime() {
OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
bloodbank + - [ ] [ ] ... >
rashmeetkaur@Rashmeets-MacBook-Air DSA FINAL PROJECT % ./bloodbank
16. View Emergency Queue Dashboard
17. Display Network Connections
Enter Choice: 2

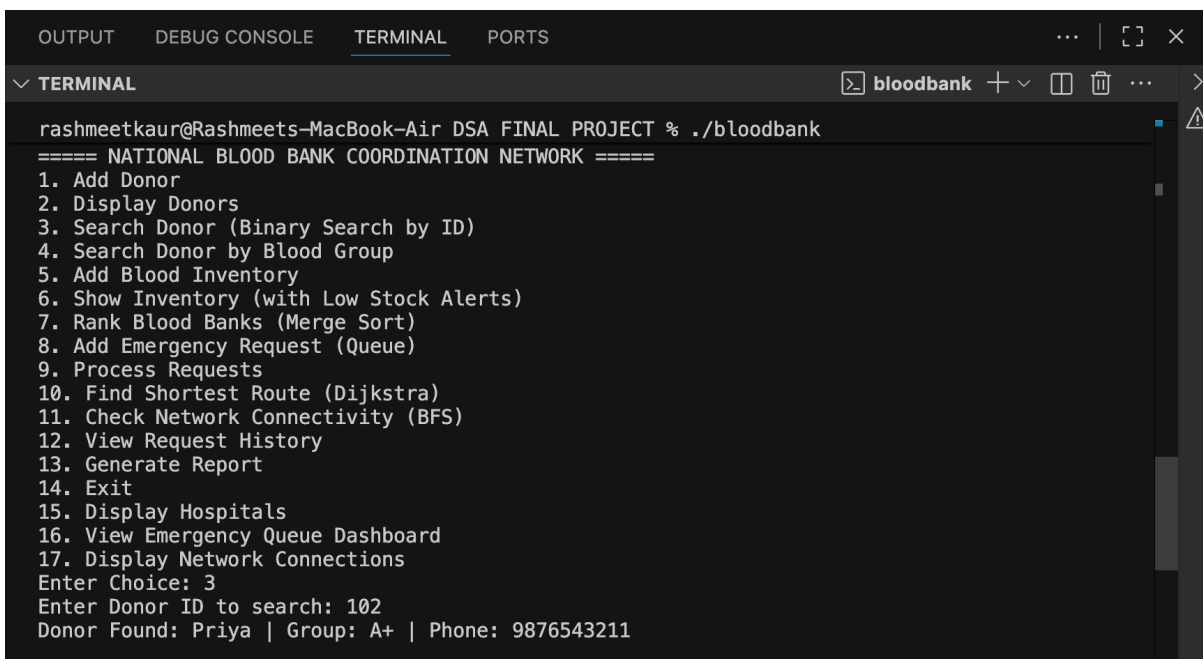
--- Donor List ---
ID      Name      Blood Group  Phone      Status
101     Rahul      0+          9876543210 Available
102     Priya      A+          9876543211 Available
103     Aman       B-          9876543212 Available
104     Simran     0-          9876543213 Available
105     Karan      AB+         9876543214 Available

===== NATIONAL BLOOD BANK COORDINATION NETWORK =====
1. Add Donor
2. Display Donors
3. Search Donor (Binary Search by ID)
4. Search Donor by Blood Group
5. Add Blood Inventory
6. Show Inventory (with Low Stock Alerts)
7. Rank Blood Banks (Merge Sort)
8. Add Emergency Request (Queue)
9. Process Requests
```

Description

The Donor Registry module stores and displays information about registered blood donors. Each donor record includes donor ID, donor name, blood group, contact information, and availability status. This information is essential for locating compatible donors during blood shortages and emergency situations.

Figure 4 :- Binary Search Donor Lookup



```
OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
bloodbank + - [ ] [ ] ... >
rashmeetkaur@Rashmeets-MacBook-Air DSA FINAL PROJECT % ./bloodbank
===== NATIONAL BLOOD BANK COORDINATION NETWORK =====
1. Add Donor
2. Display Donors
3. Search Donor (Binary Search by ID)
4. Search Donor by Blood Group
5. Add Blood Inventory
6. Show Inventory (with Low Stock Alerts)
7. Rank Blood Banks (Merge Sort)
8. Add Emergency Request (Queue)
9. Process Requests
10. Find Shortest Route (Dijkstra)
11. Check Network Connectivity (BFS)
12. View Request History
13. Generate Report
14. Exit
15. Display Hospitals
16. View Emergency Queue Dashboard
17. Display Network Connections
Enter Choice: 3
Enter Donor ID to search: 102
Donor Found: Priya | Group: A+ | Phone: 9876543211
```

Description

This module demonstrates the implementation of the Binary Search Algorithm for efficient donor retrieval. The system searches donors using their unique donor ID from a sorted donor database. Binary Search significantly reduces search time compared to linear searching, making donor identification faster and more efficient.

Algorithm Used :- Binary Search

Time Complexity :- $O(\log n)$

Figure 5 :- Blood Group Search

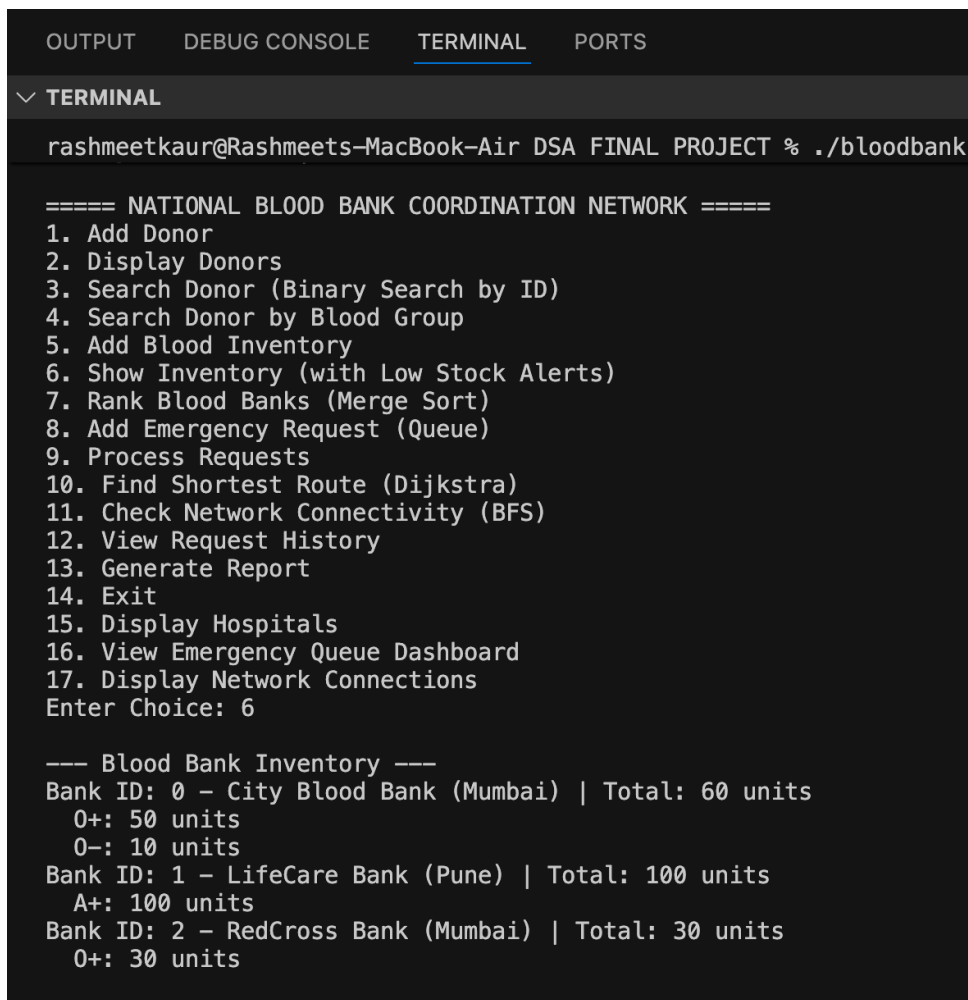
```
===== NATIONAL BLOOD BANK COORDINATION NETWORK =====
1. Add Donor
2. Display Donors
3. Search Donor (Binary Search by ID)
4. Search Donor by Blood Group
5. Add Blood Inventory
6. Show Inventory (with Low Stock Alerts)
7. Rank Blood Banks (Merge Sort)
8. Add Emergency Request (Queue)
9. Process Requests
10. Find Shortest Route (Dijkstra)
11. Check Network Connectivity (BFS)
12. View Request History
13. Generate Report
14. Exit
15. Display Hospitals
16. View Emergency Queue Dashboard
17. Display Network Connections
Enter Choice: 4
Enter Blood Group: 0+

Available Donors for 0+:
- Rahul (ID: 101) | Contact: 9876543210
```

Description

This module allows searching donors based on blood group requirements. The system scans the donor database and displays all donors matching the requested blood group. This functionality helps healthcare authorities quickly identify eligible donors during emergency situations.

Figure 6 :- Blood Inventory Dashboard



```
OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
▼ TERMINAL
rashmeetkaur@Rashmeets-MacBook-Air DSA FINAL PROJECT % ./bloodbank

===== NATIONAL BLOOD BANK COORDINATION NETWORK =====
1. Add Donor
2. Display Donors
3. Search Donor (Binary Search by ID)
4. Search Donor by Blood Group
5. Add Blood Inventory
6. Show Inventory (with Low Stock Alerts)
7. Rank Blood Banks (Merge Sort)
8. Add Emergency Request (Queue)
9. Process Requests
10. Find Shortest Route (Dijkstra)
11. Check Network Connectivity (BFS)
12. View Request History
13. Generate Report
14. Exit
15. Display Hospitals
16. View Emergency Queue Dashboard
17. Display Network Connections
Enter Choice: 6

---- Blood Bank Inventory ----
Bank ID: 0 - City Blood Bank (Mumbai) | Total: 60 units
    O+: 50 units
    O-: 10 units
Bank ID: 1 - LifeCare Bank (Pune) | Total: 100 units
    A+: 100 units
Bank ID: 2 - RedCross Bank (Mumbai) | Total: 30 units
    O+: 30 units
```

Description

The Blood Inventory Dashboard displays blood stock available across all registered blood banks. It helps healthcare

administrators monitor inventory levels, identify shortages, and ensure adequate blood availability for emergency requests.

Figure 7 :- Blood Bank Ranking (Merge Sort)

```
===== NATIONAL BLOOD BANK COORDINATION NETWORK =====
1. Add Donor
2. Display Donors
3. Search Donor (Binary Search by ID)
4. Search Donor by Blood Group
5. Add Blood Inventory
6. Show Inventory (with Low Stock Alerts)
7. Rank Blood Banks (Merge Sort)
8. Add Emergency Request (Queue)
9. Process Requests
10. Find Shortest Route (Dijkstra)
11. Check Network Connectivity (BFS)
12. View Request History
13. Generate Report
14. Exit
15. Display Hospitals
16. View Emergency Queue Dashboard
17. Display Network Connections
Enter Choice: 7
Blood Banks Ranked by Total Inventory:
ID: 1 - LifeCare Bank: 100 units
ID: 0 - City Blood Bank: 60 units
ID: 2 - RedCross Bank: 30 units
```

Description

This module implements the Merge Sort Algorithm to rank blood banks according to their available inventory. Ranking helps healthcare authorities prioritize blood allocation by identifying blood banks with the highest stock levels.

Figure 8 :- Transportation Network Connections

```
===== NATIONAL BLOOD BANK COORDINATION NETWORK =====
1. Add Donor
2. Display Donors
3. Search Donor (Binary Search by ID)
4. Search Donor by Blood Group
5. Add Blood Inventory
6. Show Inventory (with Low Stock Alerts)
7. Rank Blood Banks (Merge Sort)
8. Add Emergency Request (Queue)
9. Process Requests
10. Find Shortest Route (Dijkstra)
11. Check Network Connectivity (BFS)
12. View Request History
13. Generate Report
14. Exit
15. Display Hospitals
16. View Emergency Queue Dashboard
17. Display Network Connections
Enter Choice: 17

--- Transportation Network Connections ---
City Blood Bank <--> City Hospital (5 km)
City Blood Bank <--> LifeCare Bank (15 km)
City Blood Bank <--> RedCross Bank (4 km)
LifeCare Bank <--> Grace Clinic (3 km)
RedCross Bank <--> City Hospital (2 km)
```

Description

This module displays the transportation network connecting blood banks and hospitals. The network is represented as a graph where facilities act as nodes and transportation routes act as edges. This structure supports route optimization and connectivity analysis.

Figure 9 :- Dijkstra Route Optimization

```
===== NATIONAL BLOOD BANK COORDINATION NETWORK =====
1. Add Donor
2. Display Donors
3. Search Donor (Binary Search by ID)
4. Search Donor by Blood Group
5. Add Blood Inventory
6. Show Inventory (with Low Stock Alerts)
7. Rank Blood Banks (Merge Sort)
8. Add Emergency Request (Queue)
9. Process Requests
10. Find Shortest Route (Dijkstra)
11. Check Network Connectivity (BFS)
12. View Request History
13. Generate Report
14. Exit
15. Display Hospitals
16. View Emergency Queue Dashboard
17. Display Network Connections
Enter Choice: 10
Select Source Node ID (0-4):
0-2: Blood Banks, 3-4: Hospitals
0
Select Destination Node ID (0-4): 4
Shortest Distance: 18 km
Path: City Blood Bank -> LifeCare Bank -> Grace Clinic
```

Description

This module demonstrates the implementation of Dijkstra's Algorithm for finding the shortest transportation route between healthcare facilities. The algorithm minimizes travel distance and supports rapid blood delivery during emergency situations.

Figure 10 :- BFS Connectivity Verification

```
===== NATIONAL BLOOD BANK COORDINATION NETWORK =====
```

1. Add Donor
2. Display Donors
3. Search Donor (Binary Search by ID)
4. Search Donor by Blood Group
5. Add Blood Inventory
6. Show Inventory (with Low Stock Alerts)
7. Rank Blood Banks (Merge Sort)
8. Add Emergency Request (Queue)
9. Process Requests
10. Find Shortest Route (Dijkstra)
11. Check Network Connectivity (BFS)
12. View Request History
13. Generate Report
14. Exit
15. Display Hospitals
16. View Emergency Queue Dashboard
17. Display Network Connections

```
Enter Choice: 11
```

```
Network Status: ALL FACILITIES CONNECTED.
```

Description

This module uses Breadth First Search (BFS) to verify network connectivity. BFS traverses all healthcare facilities level by level and ensures that every blood bank and hospital remains connected within the transportation network.

Figure 11 :- Emergency Request Creation

```
===== NATIONAL BLOOD BANK COORDINATION NETWORK =====  
1. Add Donor  
2. Display Donors  
3. Search Donor (Binary Search by ID)  
4. Search Donor by Blood Group  
5. Add Blood Inventory  
6. Show Inventory (with Low Stock Alerts)  
7. Rank Blood Banks (Merge Sort)  
8. Add Emergency Request (Queue)  
9. Process Requests  
10. Find Shortest Route (Dijkstra)  
11. Check Network Connectivity (BFS)  
12. View Request History  
13. Generate Report  
14. Exit  
15. Display Hospitals  
16. View Emergency Queue Dashboard  
17. Display Network Connections  
Enter Choice: 8  
Enter Hospital Name: City Hospital  
Enter Blood Group Required: O+  
Enter Units: 20  
Request Added to Emergency Queue!
```

Description

This module allows hospitals to submit emergency blood requests. Each request contains hospital information, required blood group, and the number of blood units needed. The request is stored in a queue for systematic processing.

Figure 12 : – Emergency Queue Dashboard


```
===== NATIONAL BLOOD BANK COORDINATION NETWORK =====
1. Add Donor
2. Display Donors
3. Search Donor (Binary Search by ID)
4. Search Donor by Blood Group
5. Add Blood Inventory
6. Show Inventory (with Low Stock Alerts)
7. Rank Blood Banks (Merge Sort)
8. Add Emergency Request (Queue)
9. Process Requests
10. Find Shortest Route (Dijkstra)
11. Check Network Connectivity (BFS)
12. View Request History
13. Generate Report
14. Exit
15. Display Hospitals
16. View Emergency Queue Dashboard
17. Display Network Connections
Enter Choice: 16

--- Emergency Response Dashboard ---
1. City Hospital | 0+ | 20 Units
```

Description

The Emergency Queue Dashboard displays all pending blood requests currently waiting for processing. Requests are maintained in First-In-First-Out (FIFO) order using a Queue Data Structure to ensure fair and organized handling of emergencies.

Figure 13 :- Emergency Request Processing

```
===== NATIONAL BLOOD BANK COORDINATION NETWORK =====
1. Add Donor
2. Display Donors
3. Search Donor (Binary Search by ID)
4. Search Donor by Blood Group
5. Add Blood Inventory
6. Show Inventory (with Low Stock Alerts)
7. Rank Blood Banks (Merge Sort)
8. Add Emergency Request (Queue)
9. Process Requests
10. Find Shortest Route (Dijkstra)
11. Check Network Connectivity (BFS)
12. View Request History
13. Generate Report
14. Exit
15. Display Hospitals
16. View Emergency Queue Dashboard
17. Display Network Connections
Enter Choice: 9
Processing Request for City Hospital (20 units of O+)...
Allocated from City Blood Bank. Remaining stock: 30 units.
```

Description

This module processes emergency blood requests from the queue. Requests are handled according to FIFO principles, and blood units are allocated from suitable blood banks with sufficient inventory. The allocation details are recorded for future reference.

Figure 14 :- Request History

```
===== NATIONAL BLOOD BANK COORDINATION NETWORK =====
1. Add Donor
2. Display Donors
3. Search Donor (Binary Search by ID)
4. Search Donor by Blood Group
5. Add Blood Inventory
6. Show Inventory (with Low Stock Alerts)
7. Rank Blood Banks (Merge Sort)
8. Add Emergency Request (Queue)
9. Process Requests
10. Find Shortest Route (Dijkstra)
11. Check Network Connectivity (BFS)
12. View Request History
13. Generate Report
14. Exit
15. Display Hospitals
16. View Emergency Queue Dashboard
17. Display Network Connections
Enter Choice: 12

--- Request History (Completed) ---

Date/Time: Tue Jun 23 15:16:25 2026
Hospital: City Hospital
Group: 0+
Units: 20 [COMPLETED]
```

Description

The Request History module maintains records of all completed emergency blood requests. Historical records help healthcare administrators analyze system performance, monitor blood utilization, and generate operational reports.

Figure 15 :- Healthcare Analytics Report

```
===== NATIONAL BLOOD BANK COORDINATION NETWORK =====
1. Add Donor
2. Display Donors
3. Search Donor (Binary Search by ID)
4. Search Donor by Blood Group
5. Add Blood Inventory
6. Show Inventory (with Low Stock Alerts)
7. Rank Blood Banks (Merge Sort)
8. Add Emergency Request (Queue)
9. Process Requests
10. Find Shortest Route (Dijkstra)
11. Check Network Connectivity (BFS)
12. View Request History
13. Generate Report
14. Exit
15. Display Hospitals
16. View Emergency Queue Dashboard
17. Display Network Connections
Enter Choice: 13

--- National Blood Bank Report ---
Total Blood Banks: 3
Total Donors: 5
Available Donors: 5
Total Completed Requests: 1
Pending Emergency Requests: 0
Total Blood Units in Network: 170
Low Stock Alerts (Blood Groups): 0
Most Requested Blood Group: 0+
```

Description

This module generates a comprehensive healthcare analytics report containing information about blood banks, donors, emergency requests, inventory levels, low stock alerts, and blood utilization statistics. The report supports data-driven decision making and resource planning.

Figure 16 :- System Exit Message

```
===== NATIONAL BLOOD BANK COORDINATION NETWORK =====
1. Add Donor
2. Display Donors
3. Search Donor (Binary Search by ID)
4. Search Donor by Blood Group
5. Add Blood Inventory
6. Show Inventory (with Low Stock Alerts)
7. Rank Blood Banks (Merge Sort)
8. Add Emergency Request (Queue)
9. Process Requests
10. Find Shortest Route (Dijkstra)
11. Check Network Connectivity (BFS)
12. View Request History
13. Generate Report
14. Exit
15. Display Hospitals
16. View Emergency Queue Dashboard
17. Display Network Connections
Enter Choice: 14

Thank You For Using National Blood Bank Coordination Network!
rashmeetkaur@Rashmeets-MacBook-Air DSA FINAL PROJECT %
```

Ln 15, Col 30 Spaces: 4 UTF-8 LF

Description

This module safely terminates the National Blood Bank Coordination Network application. Before exiting, the system completes all active operations and displays a confirmation message indicating successful program termination.

RESULTS AND FINDINGS

15.1 Project Outcomes

The National Blood Bank Coordination Network was successfully designed and implemented using C++ and various Data Structures and Algorithms. The developed system effectively simulates a real-world healthcare logistics platform that manages blood inventories, donor records, hospital requests, transportation networks, and emergency response operations.

The project demonstrates how computational techniques can be applied to solve critical healthcare resource management problems. Through the implementation of Queue, Binary Search, Merge Sort, Dijkstra's Algorithm, and Breadth First Search (BFS), the system provides efficient management of blood resources and supports timely medical assistance.

The application successfully integrates multiple healthcare modules into a unified system capable of handling inventory monitoring, donor coordination, emergency allocation, route optimization, and healthcare analytics.

15.2 Functional Results

The developed system successfully achieved all functional requirements specified in the case study.

Donor Management

The donor registry module successfully stores and manages donor information including donor ID, donor name, blood group, contact details, and availability status.

The system allows :-

- Adding new donors
- Displaying donor records
- Searching donors by ID
- Searching donors by blood group

The donor database remains organized and supports rapid retrieval of donor information during emergencies.

Blood Inventory Management

The blood inventory module successfully tracks blood units available across multiple blood banks.

The system provides:

- Inventory monitoring
- Blood stock updates
- Inventory visualization
- Low stock identification

This functionality ensures better resource planning and minimizes the risk of blood shortages.

Emergency Request Handling

The Queue Data Structure was successfully implemented for emergency request processing.

Results show that:

- Requests are stored efficiently.
- Requests are processed in FIFO order.
- Emergency operations remain organized.

- Hospitals receive systematic request handling.

The queue-based approach ensures fairness and prevents request conflicts.

Blood Bank Ranking

Merge Sort was successfully implemented to rank blood banks based on available inventory.

The ranking system enables :-

- Quick identification of high-stock facilities.
- Better blood allocation decisions.
- Efficient emergency planning.

The algorithm maintained $O(n \log n)$ performance while sorting inventory records.

Donor Search Efficiency

Binary Search was implemented to locate donor records using donor IDs.

Results indicate :-

- Faster search performance.
- Reduced lookup time.
- Efficient handling of larger donor databases.

Compared to sequential searching, Binary Search significantly improves retrieval efficiency.

Route Optimization

Dijkstra's Algorithm successfully calculated shortest transportation routes between healthcare facilities.

The system was able to :-

- Determine optimal delivery paths.
- Calculate shortest travel distances.
- Improve transportation planning.

This functionality is critical in emergency situations where rapid blood delivery can save lives.

Network Connectivity Verification

Breadth First Search (BFS) was successfully implemented to verify connectivity among blood banks and hospitals.

Results showed :-

- All healthcare facilities were reachable.
- No isolated nodes were detected.
- Transportation reliability was maintained.

The BFS implementation strengthens network reliability and supports uninterrupted healthcare operations.

15.3 Performance Analysis

The project demonstrates efficient algorithmic performance.

Operation	Complexity
Add Donor	$O(1)$
Search Donor	$O(\log n)$
Blood Group Search	$O(n)$
Add Request	$O(1)$
Process Request	$O(1)$
Merge Sort Ranking	$O(n \log n)$
Dijkstra Route Search	$O((V+E) \log V)$
BFS Connectivity Check	$O(V+E)$

The selected algorithms provide efficient performance even when the number of donors, blood banks, hospitals, and requests increases.

15.4 Key Findings

The implementation produced several important findings :-

- Queue structures simplify emergency request processing.
- Binary Search drastically improves donor lookup efficiency.
- Merge Sort provides reliable inventory ranking.
- Graph-based modeling accurately represents transportation networks.
- Dijkstra's Algorithm minimizes blood delivery time.
- BFS effectively verifies healthcare connectivity.

- Centralized healthcare resource management improves operational efficiency.

15.5 Overall Achievement

The project successfully fulfills all objectives defined in the National Blood Bank Coordination Network case study.

The developed system demonstrates the practical application of Data Structures and Algorithms in healthcare logistics and serves as an effective simulation of a real-world blood management platform.

FUTURE SCOPE

16.1 Introduction

Although the current implementation successfully fulfills the requirements of the case study, several enhancements can further improve functionality, scalability, and real-world applicability.

Future versions of the system can transform the project into a complete healthcare resource management platform capable of supporting large-scale national blood bank operations.

16.2 Database Integration

Currently, donor information, inventory records, and request data are stored in memory during program execution.

Future implementations can integrate :-

- MySQL

- PostgreSQL
- MongoDB

This would provide :-

- Permanent data storage
- Better scalability
- Faster data management
- Enterprise-level reliability

16.3 Web-Based Dashboard

The present system operates through a console-based interface.

Future versions may include :-

- Web dashboards
- Interactive analytics
- Real-time monitoring
- Administrative portals

This would improve usability and accessibility.

16.4 Mobile Application Integration

A dedicated mobile application can be developed for :-

- Donor registration
- Blood request submission
- Emergency notifications
- Inventory tracking

Mobile access would significantly improve communication between donors, hospitals, and blood banks.

16.5 Real-Time GPS Tracking

Future transportation modules can integrate:

- GPS systems
- Google Maps APIs
- Real-time route monitoring

Benefits include :-

- Faster deliveries
- Dynamic route optimization
- Reduced transportation delays

16.6 Artificial Intelligence Integration

AI-based modules can predict :-

- Blood demand patterns
- Inventory shortages
- Donation trends
- Emergency requirements

Machine learning models could help optimize inventory management and reduce wastage.

16.7 Cloud Deployment

Future implementations can utilize cloud platforms such as :-

- AWS
- Microsoft Azure
- Google Cloud Platform

Cloud deployment would provide :-

- Scalability
- Data backup
- High availability
- Remote accessibility

16.8 National Healthcare Integration

The system can be integrated with:

- Government healthcare portals
- Hospital information systems
- National donor registries
- Emergency response centers

This would create a unified healthcare coordination platform at a national level.

16.9 Advanced Analytics

Future versions may include:

- Predictive analytics
- Blood utilization forecasting
- Performance dashboards
- Data visualization reports

These features would support strategic healthcare planning.

16.10 Future Vision

The long-term vision is to transform the National Blood Bank Coordination Network into a nationwide intelligent healthcare platform capable of supporting millions of donors, thousands of

hospitals, and large-scale emergency operations while ensuring timely blood availability and efficient healthcare logistics.

CONCLUSION

The National Blood Bank Coordination Network successfully demonstrates the practical application of Data Structures and Algorithms in solving real-world healthcare logistics challenges.

Blood is one of the most critical medical resources required for surgeries, trauma treatment, cancer therapy, emergency response, and disaster management. Efficient coordination between blood banks, hospitals, and donors is essential to ensure that blood reaches patients whenever and wherever it is needed. Managing these resources manually can lead to delays, inefficiencies, and increased risks during emergencies.

To address these challenges, the project was designed as a centralized healthcare coordination system capable of managing donor registries, blood inventories, emergency requests, transportation networks, and healthcare analytics. The implementation combines multiple Data Structures and Algorithms to create an efficient and scalable solution.

The Queue Data Structure was successfully utilized for emergency request management, ensuring fair and systematic request processing based on FIFO principles. Binary Search provided efficient donor retrieval from sorted donor databases, significantly

reducing search time. Merge Sort enabled accurate ranking of blood banks according to inventory availability, supporting effective allocation decisions. Dijkstra's Algorithm optimized transportation routes between healthcare facilities, reducing delivery times during emergencies. Breadth First Search verified network connectivity and ensured that all healthcare facilities remained connected within the transportation network.

The project also incorporated additional features such as low-stock alerts, request history tracking, healthcare analytics reporting, donor availability management, transportation network visualization, and emergency response dashboards. These features collectively improve operational efficiency and support healthcare decision-making.

From an academic perspective, this project strengthened understanding of algorithm design, graph processing, object-oriented programming, complexity analysis, and healthcare information systems. It provided practical experience in applying theoretical Data Structures and Algorithms to solve real-world problems.

The successful implementation of the National Blood Bank Coordination Network confirms that appropriate selection of data structures and algorithms can significantly improve healthcare resource management. The project not only fulfills all requirements specified in the case study but also demonstrates how computational techniques can contribute to life-saving medical operations.

In conclusion, the National Blood Bank Coordination Network serves as a comprehensive demonstration of DSA concepts in healthcare logistics and stands as a practical, efficient, scalable,

and academically valuable software solution for modern blood management systems.

REFERENCES

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [2] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2011.
- [3] M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2013.
- [4] E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [5] B. Stroustrup, *The C++ Programming Language*, 4th ed. Upper Saddle River, NJ, USA: Addison-Wesley Professional, 2013.
- [6] S. S. Skiena, *The Algorithm Design Manual*, 3rd ed. Cham, Switzerland: Springer, 2020.
- [7] N. Wirth, *Algorithms + Data Structures = Programs*. Englewood Cliffs, NJ, USA: Prentice Hall, 1976.
- [8] “C++ Reference Documentation,” cppreference.com.
- [9] “Queue Container Adapter in C++,” [cppreference Queue Documentation](http://cppreference.com/queue).
- [10] “Priority Queue Container Adapter in C++,” [cppreference Priority Queue Documentation](http://cppreference.com/priority_queue).
- [11] “Map Container in C++ STL,” [cppreference Map Documentation](http://cppreference.com/map).
- [12] ITM Skills University, *Data Structures and Algorithms with C++ – Semester II Case Study Problem Statements*, B.Tech CSE 2025–2029.

GITHUB REPOSITORY

The complete source code for the **National Blood Bank Coordination Network** project, including implementation files, sample inputs, sample outputs, and project documentation, is available in the following GitHub repository.

GitHub Repository

<https://github.com/krashmeet033-sys/DSA-FINAL-PROJECT->

Repository Contents

The repository contains :-

- Complete C++ implementation of the National Blood Bank Coordination Network
- main.cpp containing all core functionalities and algorithm implementations
- Blood Bank Inventory Management System
- Donor Registry and Search Engine
- Hospital Database Module
- Emergency Request Queue Processing System
- Blood Inventory Ranking using Merge Sort
- Donor Search using Binary Search
- Transportation Network Graph Implementation
- Route Optimization using Dijkstra's Algorithm
- Network Connectivity Verification using BFS

- Healthcare Analytics and Reporting Module
- sample_input.txt for automated project testing
- sample_output.txt demonstrating expected program output
- README.md containing project description, objectives, architecture, algorithms, and execution instructions
- GitHub project documentation and project report references

Repository Features

The repository demonstrates practical implementation of the following Data Structures and Algorithms :-

Algorithm / Data Structure	Purpose
Queue	Emergency Request Processing
Vector	Storage of Donors, Hospitals and Blood Banks
Map	Blood Inventory Management
Binary Search	Fast Donor Search
Merge Sort	Blood Bank Ranking
Graph	Transportation Network Representation
Dijkstra's Algorithm	Shortest Route Optimization
BFS	Network Connectivity Verification

Repository Statistics

Project Title :-

National Blood Bank Coordination Network

Programming Language :-

C++

Development Environment :-

Visual Studio Code

Compiler :-

G++

Project Type :-

Menu-Driven Console Application

Domain :-

Healthcare Logistics and Blood Resource Management

Institution :-

ITM Skills University

Course :-

Data Structures and Algorithms with C++

Academic Year :-

2025 – 2029

Final Project Deliverables Included

- ✓ Source Code
- ✓ Algorithm Implementations
- ✓ Sample Input File
- ✓ Sample Output File
- ✓ GitHub Repository
- ✓ README Documentation

- ✓ Project Report
- ✓ Complexity Analysis
- ✓ Screenshots of Execution
- ✓ DSA-Based Healthcare Solution

